

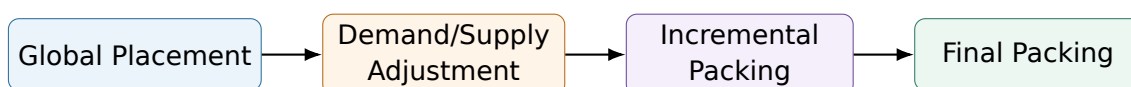
AMF-Placer

第五天学习笔记

Resource Adjustment、Incremental/Final Packing 与实验闭环

今天的目标不是再引入一个大型优化公式，而是回答：

当 analytical placement 已经把线长、density 和 macro legalization 逐渐推向收敛以后，AMF-Placer 如何把“连续/粗粒度位置”真正转化为满足 CLB 资源规则的 packing 结果？这些技术在实验中分别解决了什么问题？论文又有哪些明确的局限？



本日覆盖范围

- Section II-C: Resource Demand/Supply Adjustment、Incremental Packing、Final Packing 在全流程中的位置
- Section III-E: Final Packing 的并行框架与 3 个 mixed-size 修改
- Table I: benchmark 的规模与 macroRatio
- Table II: Tech1-Tech5 消融实验
- Table III: 多线程加速与并行度下降原因
- Fig. 6: AMF-Placer 与 Vivado placement 的可视化，以及论文明确承认的 timing 局限

目录

1	先把 Day 5 放回完整流程	2
2	Resource Demand/Supply Adjustment: 论文到底告诉了我们什么?	3
3	为什么 placement 里还要插入 Incremental Packing?	4
4	Final Packing: 从“坐标”变成“真实 site 内容”	5
5	论文原文: Final Packing 的基础并行框架	6
6	Mixed-size 修改 1: Macro 已经预打包, 也不等于整个 CLB 被封死	7
7	Mixed-size 修改 2: Hash Candidate Packing Cluster	8
8	Mixed-size 修改 3: 逐步扩大 Ripping-up Window	9
9	为什么这些 packing 机制必须和前面的 GP 联动?	10
10	实验背景: 这些 benchmark 真的是 mixed-size	11
11	Table II: 不要只看“提升百分比”, 要看每个 Tech 解决什么	12
12	从消融实验看前四天学过的逻辑是否成立	13
	12.1 Tech1: 好的 initial placement 对层次化设计尤其关键	13
	12.2 Tech2 / Tech3: 稳定 QP 与避免 spreading 过度扩张	13
	12.3 Tech4: 可能更慢一点, 但保护 compact placement 的 HPWL	13
	12.4 Tech5: 大型 BRAM macro 对 progressive legalization 很敏感	13
13	Table III: 为什么 8 线程不是 8 倍加速?	14
14	Fig. 6: 能进 Vivado routing, 但 timing 不是本文强项	15
15	把整篇论文串成一条因果链	16
16	Day 5 自测与下一步	17

1 先把 Day 5 放回完整流程

前四天我们已经学习了：SA-based initial placement、quadratic placement、cell spreading，以及 progressive macro legalization。今天学习的是 placement 后半段如何“收口”。

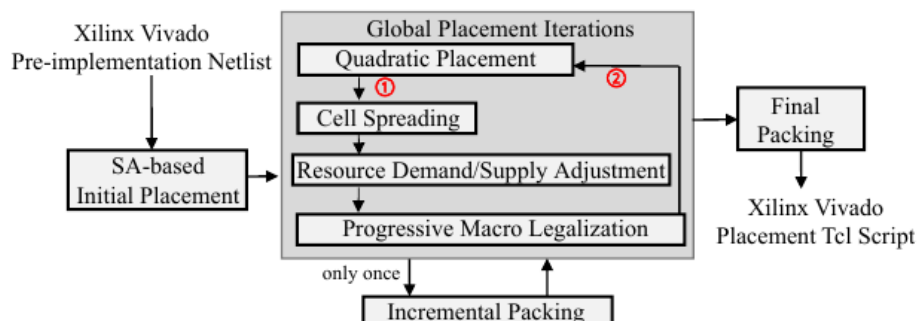


Fig. 3. The Outline of AMF-Placer's Workflow with 7 Process Phases: Label 1 indicates lower-bound placement and label 2 indicates upper-bound placer.

4) *Resource Demand/Supply Adjustment*: Based on the packing feasibility and routing congestion level, the area

图 1: 论文 Fig. 3: AMF-Placer 的 7 个阶段。图像直接截取自原论文。

论文对三个阶段的定义非常明确：

- **Resource Demand/Supply Adjustment**: 根据 packing feasibility 与 routing congestion，调整 LUT/FF 等 standard cells 的 area demand，以及部分区域的 area supply；
- **Incremental Packing**: 在 global placement 中，将部分 LUT/FF 配成 LUT-FF macro 或 FF-FF macro，以缩小问题规模并尽早识别 CLB internal nets；
- **Final Packing**: global placement 迭代结束后，将 instances 映射到具有固定资源类型和数量的物理 sites，作为最终 legalization / detailed placement。

今天最重要的一条主线

Global placement 解决的是“大致应该在哪里”；packing 解决的是“这些逻辑在一个真实 FPGA site 内到底能不能装得下、如何装进去”。

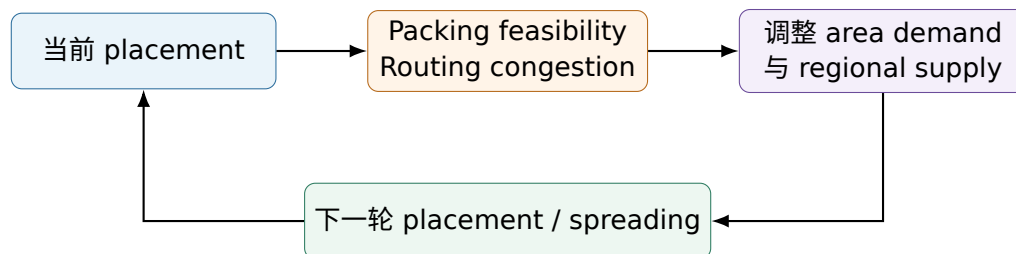
2 Resource Demand/Supply Adjustment: 论文到底告诉了我们什么?

这一部分必须特别谨慎。AMF-Placer 本文没有给出完整的 **demand/supply adjustment** 算法和公式。论文只给出以下信息：

1. 调整依据是 **packing feasibility** 和 **routing congestion level**;
2. LUT、FF 等 standard cells 的 area demand 可能增加或减少;
3. 某些区域的 area supply 也可能增加或减少;
4. cell spreading 每轮之后会继续根据 packing / routing feasibility 调整 demand 和 supply;
5. 这一机制来自 extended UTPlaceF [19], AMF-Placer 对其做了 mixed-size placement 适配。

因此，我们可以把论文中支持的反馈关系写成：

packing/routing feedback → demand/supply correction → 下一轮 spreading / placement



论文边界

本文没有说明“某种 congestion 对应多少 demand inflation”“区域 supply 具体如何数值化”等细节。若要继续追到公式和实现，必须再读论文引用的 extended UTPlaceF [19]。本笔记不自行补出本文没有给出的公式。

3 为什么 placement 里还要插入 Incremental Packing?

论文在 global placement 期间只做一种相对轻量的 packing:

$$\text{LUT} + \text{FF} \rightarrow \text{LUT-FF macro}, \quad \text{FF} + \text{FF} \rightarrow \text{FF-FF macro}.$$

它的两个论文原文目的分别是:

1. **shrink the problem size;**
2. **identify CLB internal nets at early stage**, 从而改善 placement quality。



论文没有展开的地方

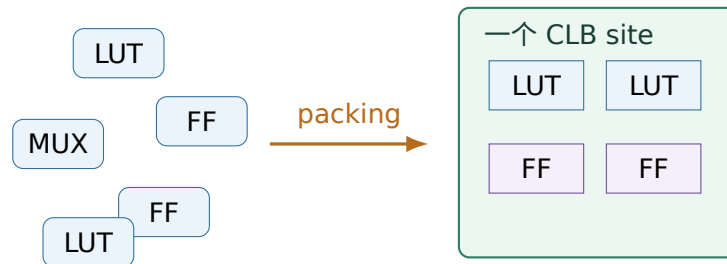
AMF-Placer 只说明 incremental packing 的 pairing algorithm 采用 RippleFPGA [9]。本文没有重新给出其 pair score、兼容性规则或候选搜索细节，因此 Day 5 只把它作为流程组件理解，不虚构内部评分函数。

4 Final Packing: 从“坐标”变成“真实 site 内容”

Global placement 最终给我们的仍然主要是 instance 的位置倾向。Final packing 要完成的是：

instances near a CLB → a legal combination inside that CLB site

对于 CLB，必须考虑 LUT、FF、MUX、CARRY 等固定数量和类型的 BEL 资源。论文把 Final Packing 称为 final legalization / detailed placement 的组成部分。



辅助理解：packing 与 placement 的差异

这张示意图是为了帮助理解术语，不是论文中的硬件结构图。关键点是：placement 更关注全局几何位置；packing 必须服从 site 内部的离散资源与兼容性规则。

5 论文原文: Final Packing 的基础并行框架

column legalization procedures for different columns can be parallelized (line 17-21 in Algorithm 2).

E. Final Packing

For the final packing, a parallelized packing algorithm with high quality has been proposed by [19]. It allows the FPGA sites to search their corresponding candidate packing solutions concurrently and then negotiate together during synchronization. It handles exception instances that cannot be packed during the parallel method with the sequential conventional ripping-up algorithm. We adapt this high-performance algorithm to our mixed-size placement scenarios with several major modifications: (1) Some instances might be pre-packed into CLB sites by the macro legalization, but these pre-packed CLBs can be filled with other instances as long as the rules described in Section II-A are obeyed; (2) We encode each candidate packing cluster for a specific site with hash function to avoid many redundant packing attempts to improve performance and quality of packing; (3) By iteratively increasing the ripping-up window size and processing independent exception instances concurrently, we also parallelize the exception handling algorithm and achieve further acceleration.

IV. EXPERIMENTAL RESULTS

A. Target Device, Benchmarks and Environment

图 2: 论文 Section III-E “Final Packing” 原文片段, 直接裁取自第 7 页。

论文沿用了 [19] 的高性能并行 packing 框架, 其基本思想是:

- 不同 FPGA sites 可以并发搜索各自的 **candidate packing solutions**;
- 到 synchronization 阶段再进行协商;
- 对并行阶段仍无法成功 packing 的 exception instances, 再使用 conventional sequential ripping-up algorithm 处理。

可以把它理解为:

parallel candidate search → synchronization / negotiation → exception repair

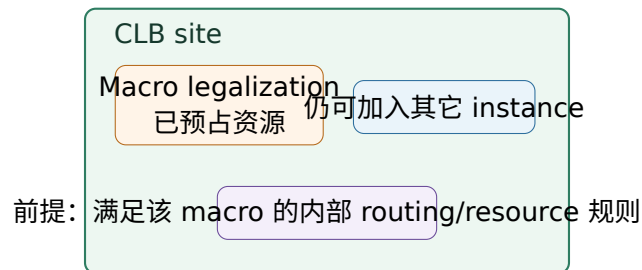
为什么需要“协商”?

并行搜索时, 不同 sites 的局部候选可能会竞争同一个 instance 或相互造成资源冲突。因此局部搜索之后还需要同步阶段来确定一致的最终选择。论文没有在本文中展开 negotiation 的具体协议。

6 Mixed-size 修改 1: Macro 已经预打包，也不等于整个 CLB 被封死

论文指出，一些 instances 可能已经在 macro legalization 阶段被 pre-packed 到 CLB sites 中。但是，只要满足 Section II-A 描述的规则，这些 pre-packed CLBs 仍可以继续容纳其它 instances。

这个细节很重要，因为 mixed-size macro 并不总是“独占一个 CLB”。论文前面就指出，一些 macros 可以在细粒度上与其它 instances 共享 CLB 资源。



不要误解 pre-packed

pre-packed 表示“某些 BEL/resource 已经被宏约束占用”，而不是自动等价于“整个 site 不再允许其它逻辑进入”。AMF-Placer 的 final packing 会继续利用剩余可合法使用的资源。

7 Mixed-size 修改 2: Hash Candidate Packing Cluster

论文第二个修改是：对“某个 site 的 candidate packing cluster”进行 hash encoding，以避免大量重复 packing attempts。

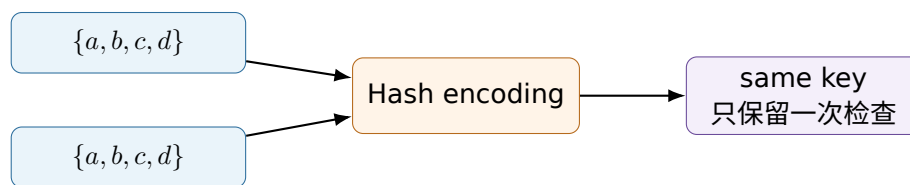
假设搜索过程中多次产生逻辑上相同的候选集合：

$$C_1 = \{a, b, c, d\}, \quad C_2 = \{a, b, c, d\}.$$

如果没有去重，placer 可能重复检查同一个候选组合。论文的做法可以抽象为：

$$\text{key}(C) = \text{Hash}(C).$$

若某个 key 已经访问过，则可以避免再次执行相同的候选尝试。



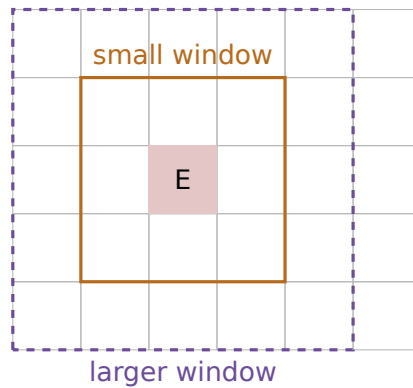
论文支持到哪一步？

论文只说明使用 hash function 编码 candidate packing cluster 来减少 redundant attempts, 并未给出具体 hash key 的字段组成、冲突处理方式或缓存数据结构。因此这些实现细节不能从本文直接得出。

8 Mixed-size 修改 3: 逐步扩大 Ripping-up Window

并行 packing 后总会有一些 exception instances 无法立即合法装入。论文的第三个修改是：

1. 逐步增加 ripping-up window size;
2. 对彼此独立的 exception instances 并发处理;
3. 因而把原本偏 sequential 的 exception handling 也进一步并行化。



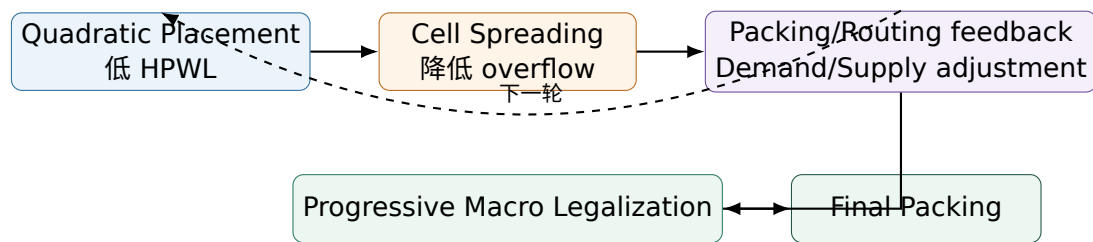
先在较小邻域内尝试
若局部无法重排成功
再扩大 ripping-up window
避免一开始扰动过大区域

辅助术语解释

在 placement/packing 文献中, ripping-up 通常指撤销一部分已有局部决定, 再重新安排它们以给难处理的 instance 腾出合法空间。这里的“逐步扩大窗口”是论文明确给出的策略; 图中的网格只是概念示意。

9 为什么这些 packing 机制必须和前面的 GP 联动?

AMF-Placer 不是把 global placement 和 packing 当作两个完全独立的黑盒。论文的流程本质上存在反馈:



这也解释了论文在实验部分提到的“vicious circle”: 如果 spreading window 不好、cell update 又过激、macro legalization 又直接发生大位移, 那么坏的 global placement 会进一步让 parallel packing 更难, 最终还会增加 packing runtime。

系统级理解

AMF-Placer 的贡献不是一个孤立公式, 而是把 initialization、wirelength optimization、spreading、macro legalization、packing 设计成一组相互稳定的机制。某一阶段产生的坏解, 可能在后续阶段被放大。

10 实验背景：这些 benchmark 真的是 mixed-size

TABLE I: BENCHMARK PARAMETERS

Benchmarks	Rosetta FaceDetection	SpoNN	OptimSoC	MiniMap2	OpenPiton	MemN2N	BLSTM	Rosetta DigitRecog
#LUT	68945	63095	186183	407586	180388	184997	118967	151636
#FF	56987	70987	248983	252624	111966	84694	54690	105580
#CARRY	4978	2091	1715	19826	1712	11528	2762	1970
#Mux	2177	217	27037	180	13696	4466	36210	4662
#LUTRAM	255	251	901	251	752	3500	1147	251
#DSP	101	165	51	528	58	312	258	1
#BRAM	141	208	218	283	147	148	812	379
#Cell	134450	137937	468150	681889	309145	289721	215101	265775
#Macro	3582	1135	21882	8746	8278	5775	14651	3061
#siteForMacro	55666	23079	89004	191263	48066	118960	171822	55754
MacroRatio	40%	16%	19%	28%	15%	41%	80%	21%

(Electronic Design Interchange Format) netlists exported from

图 3: 论文 Table I: 8 个大型 benchmark 的资源规模与 MacroRatio, 直接裁取自原论文。

论文定义:

$$\text{MacroRatio} = \frac{\text{macros 所需 sites 总数}}{\text{所有 instances 所需 sites 总数}}$$

Table I 中几个非常值得记住的例子:

- MemN2N 的 MacroRatio 为 **80%**;
- OpenPiton 为 **41%**;
- Rosetta FaceDetection 为 **40%**;
- MiniMap2 包含 **309,145 cells**、**8,278 macros**;
- OptimSoC 规模达到 **681,889 cells**。

实验环境

论文目标器件为 Xilinx VU095; AMF-Placer 使用 C++ 实现。实验平台为 Ubuntu 20.04、Intel i7-6770 (3.40 GHz, 8 logic cores) 和 32GB DDR4。由于 ISPD 2015/2016 Vivado patch 的许可限制, 论文无法取得最终 routed wirelength, 因此用 HPWL 近似最终布线线长。

11 Table II: 不要只看“提升百分比”，要看每个 Tech 解决什么

TABLE II: COMPARISON OF RUNTIME AND HPWL WITH VARIOUS BENCHMARKS AND CONFIGURATION

	Rosetta FaceDetect [33]				SpoonNN [28]				OptimSoc [31]				MiniMap2 [15]			
	HPWL	R_{hpwl}	time(s)	R_{time}	HPWL	R_{hpwl}	time(s)	R_{time}	HPWL	R_{hpwl}	time(s)	R_{time}	HPWL	R_{hpwl}	time(s)	R_{time}
proposed	446908	1.000	105	1.054	339764	1.000	129	1.000	1771538	1.000	466	1.000	1275346	1.000	558	1.000
w/o Tech1	479221	1.072	109	1.088	360567	1.061	132	1.023	9034564	5.100	871	1.870	5132073	4.024	1265	2.266
w/o Tech2	465208	1.041	131	1.315	491385	1.446	517	4.007	1841786	1.040	577	1.239	1790566	1.404	1019	1.826
w/o Tech3	487060	1.090	124	1.238	431386	1.270	240	1.857	1825819	1.031	543	1.164	1307249	1.025	626	1.121
w/o Tech4	450199	1.007	100	1.000	351649	1.035	131	1.015	2658863	1.501	525	1.126	1291565	1.013	561	1.005
w/o Tech5	511514	1.145	134	1.344	443765	1.306	138	1.067	1880431	1.061	546	1.172	1430330	1.122	708	1.268
baseline	794201	1.777	234	2.338	1865746	5.491	329	2.549	2231978	1.260	559	1.199	11886747	9.320	3539	6.339
	OpenPiton [13]				Rosetta DigitRecog [33]				MemN2N [29]				BLSTM [30]			
proposed	1139189	1.000	283	1.035	726929	1.000	254	1.097	801403	1.000	218	1.130	484650	1.000	226	1.241
w/o Tech1	6218009	5.458	350	1.280	768796	1.058	256	1.105	963416	1.202	363	1.287	492194	1.016	285	1.083
w/o Tech2	1159003	1.017	326	1.190	730071	1.004	321	1.389	916176	1.143	411	1.460	496540	1.025	364	1.383
w/o Tech3	1157479	1.016	296	1.079	773957	1.065	272	1.174	862212	1.076	343	1.216	511861	1.056	296	1.126
w/o Tech4	1212149	1.064	274	1.000	728094	1.002	231	1.000	822742	1.027	282	1.000	653010	1.347	307	1.169
w/o Tech5	1142927	1.003	301	1.098	997207	1.372	244	1.053	923670	1.153	353	1.252	722495	1.491	263	1.000
baseline	1432535	1.258	297	1.086	1047885	1.442	298	1.288	1610425	2.010	547	1.942	907383	1.872	325	1.237

TABLE III: ACCELERATION RATIOS WITH PARALLELISM

	Rosetta FaceDetect	SpoonNN	OptimSoc	MiniMap2	OpenPiton	Rosetta DigitRecog	MemN2N	BLSTM
8 threads	2.17x	2.07x	2.50x	2.86x	2.63x	2.22x	2.61x	2.23x

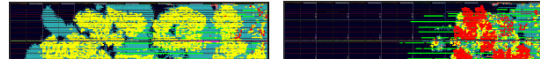


图 4: 论文 Table II / III 原始区域。Table II 是 Tech1-Tech5 消融；Table III 是并行加速。

论文五个技术分别是：

Tech	含义
Tech1	SA-based initial placement
Tech2	interconnection-density-aware pseudo-net weight
Tech3	utilization-guided spreading-window search
Tech4	forgetting-rate-based cell-spreading update
Tech5	progressive macro legalization

论文报告, 与其 baseline 相比, 完整 AMF-Placer 的 HPWL 改善范围为 **20.4%-89.3%**, runtime 降低范围为 **8.0%-84.2%**。

如何正确阅读消融

“w/o TechX” 的意义是：从完整流程中拿掉某一机制，看最终 HPWL/runtime 如何变化。它不是在比较五个技术谁 “更重要”，而是在检查每个机制是否解决了某类失稳或质量问题。

12 从消融实验看前四天学过的逻辑是否成立

12.1 Tech1: 好的 initial placement 对层次化设计尤其关键

论文指出, 去掉 Tech1 后, OptimSoC 的 normalized HPWL 变为 **5.100**, OpenPiton 为 **5.458**。作者据此强调: 对具有明显 design hierarchy 的设计, good initial placement 很关键。

12.2 Tech2 / Tech3: 稳定 QP 与避免 spreading 过度扩张

论文总结, 去掉 interconnection-density-aware pseudo weight 或 utilization-guided spreading-window search, 都会损害 placement quality 和 runtime。这和我们 Day 2/Day 3 的逻辑一致: 宏的大幅移动会破坏 quadratic approximation, 而过大的 spreading window 会引入不必要的位移。

12.3 Tech4: 可能更慢一点, 但保护 compact placement 的 HPWL

论文说明, 某些 benchmark 中去掉 Tech4 可能因为直接更新 cell location 而更快收敛, 但 compact placement 时会恶化 HPWL; 作者认为这点 runtime overhead 相比 HPWL 保护是值得的。

12.4 Tech5: 大型 BRAM macro 对 progressive legalization 很敏感

论文特别指出: 用 direct macro legalization 替换 Tech5 会恶化最终 wirelength, 尤其是具有很多大型 BRAM macros 的 SpooNN 和 BLSTM。Table II 中, BLSTM “w/o Tech5” 的 normalized HPWL 为 **1.491**, SpooNN 为 **1.306**。

最值得记住的实验结论

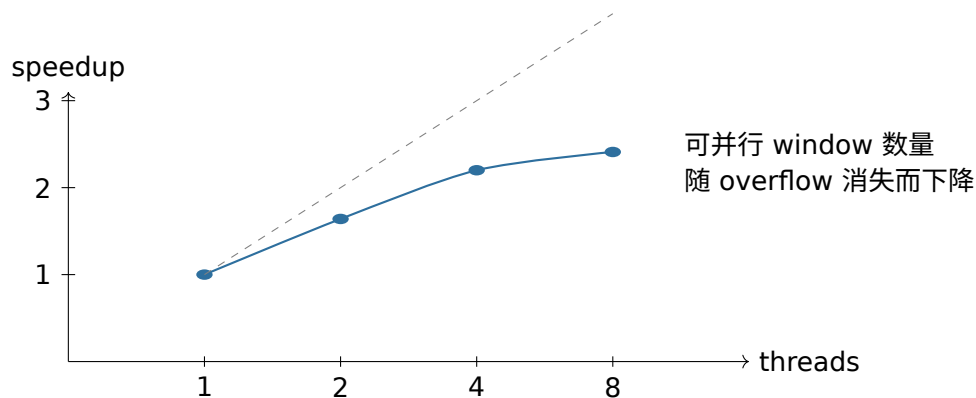
这些 Tech 并非完全独立。论文明确描述: 若没有 Tech3, spreading window 会更大; 没有 Tech4 又会进一步增加 wirelength; 再没有 Tech5, 宏 legalization 的大 displacement 会继续恶化结果, 最终还可能让 parallel packing 更困难。这是一个跨阶段的连锁效应。

13 Table III: 为什么 8 线程不是 8 倍加速?

论文报告相对单线程平均加速:

2 threads : 1.64×	4 threads : 2.20×	8 threads : 2.41×
-------------------	-------------------	-------------------

显然存在明显的 diminishing return。论文给出的原因很具体: global placement iterations 占据大部分 runtime, 而 cell spreading 中, 当多数 overflow regions 已经处理掉以后, 可同时处理的 non-overlapping spreading windows 会快速减少, 于是后期没有足够的并行任务喂给更多线程。



这里学到的是并行算法的结构限制

“算法可以 parallelize” 不等于 “线程数翻倍, 速度就翻倍”。AMF-Placer 的后期 cell spreading 暴露了典型的并行度逐步枯竭问题。

14 Fig. 6: 能进 Vivado routing, 但 timing 不是本文强项

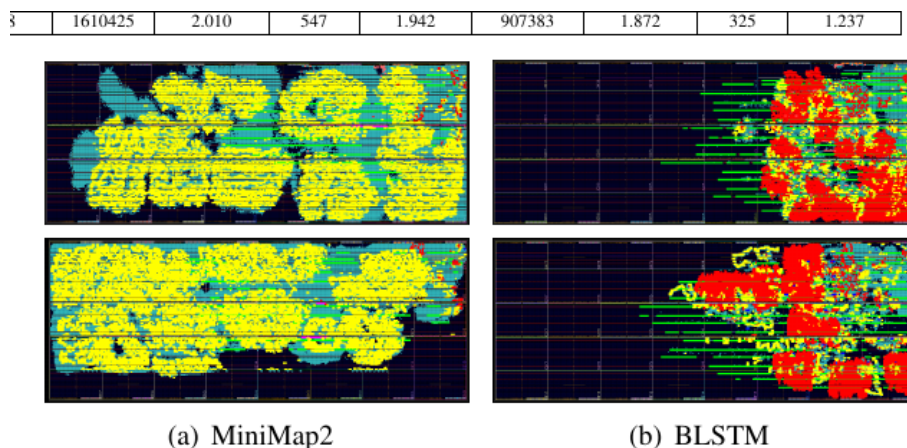


Fig. 6. Comparison of AMF-Placer Placement (upper ones) and Vivado Placement (lower ones): yellow for CARRY macros, red for MUX macros, green for BRAM macros, purple for DSP macros, blue for LUTRAM macros. The view of device is rotated left by 90°.

contests. The placement of the largest benchmark, MiniMap2 [15], and the benchmark with the highest macro ratio, BLSTM [30], are shown in Fig. 6 as examples. Different types of macros are highlighted in different colors. Due to limited space, more detailed statistics are available in the open-source

图 5: 论文 Fig. 6: MiniMap2 与 BLSTM 的 AMF-Placer (上) 和 Vivado placement (下) 对比。图中不同颜色标识不同 macro 类型。

论文给出几个非常明确的结论:

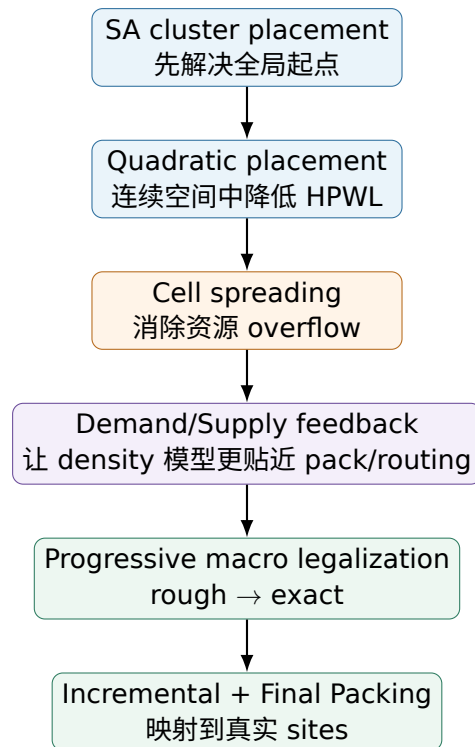
- AMF-Placer 生成的 placement 可以加载到 Vivado 进行 routing; 论文实验中的 placements 都能够成功 routed;
- 作者观察到 AMF-Placer 与 Vivado 的 congestion extent 差异较小;
- AMF-Placer 的 slice utilization 略高, 作者将其归因于 Vivado 更好的 floorplanning 和对 design hierarchy 的利用;
- **Vivado 是 timing-driven, 而 AMF-Placer 是 wirelength-driven, 因此 AMF-Placer 的 WNS/TNS 比 Vivado 差。**

这篇论文的境界必须记住

AMF-Placer 证明的是一个高性能、可扩展的 **mixed-size analytical FPGA placement** 框架, 并不是一个完整的 timing-driven commercial-quality placer。作者把 timing/floorplanning 相关改进明确留作 future work。

15 把整篇论文串成一条因果链

现在可以把 AMF-Placer 看成一个“逐步把自由度收紧”的过程：



从优化视角看

前期允许较大的连续自由度，目标是寻找低线长区域；随后逐渐加入 **density**、资源、macro shape、site packing 等离散约束。AMF-Placer 的核心不是“一开始就把所有 FPGA 约束硬塞进一个优化器”，而是用多阶段机制逐步逼近可实现解。

16 Day 5 自测与下一步

今天至少应该能回答 8 个问题

1. 为什么 AMF-Placer 既有 Incremental Packing, 又有 Final Packing?
2. Resource Demand/Supply Adjustment 的反馈信号是什么?
3. 为什么不能从本文直接写出完整的 demand inflation 公式?
4. pre-packed CLB 为什么仍可能容纳其它 instances?
5. candidate packing cluster 为什么值得做 hash 去重?
6. exception instances 为什么采用逐步扩大的 ripping-up window?
7. 为什么 8 threads 的平均 speedup 只有 $2.41\times$?
8. 为什么 AMF-Placer 的 WNS/TNS 仍然弱于 Vivado?

一页记忆版

Packing closure: GP 给位置, packing 给 site 内合法组合。

反馈: packing/routing feasibility 会反向调整 demand/supply。

Final Packing 三个 **mixed-size** 修改: pre-packed CLB 继续利用剩余资源; hash candidate clusters; 扩大 rip-up window 并并行处理独立异常。

实验: 完整系统比 baseline 显著降低 HPWL/runtime; 8-thread 平均 $2.41\times$; placement 可送入 Vivado routing。

边界: AMF-Placer 仍是 wirelength-driven, WNS/TNS 弱于 timing-driven Vivado。

建议的 Day 6

下一天不再按论文顺序继续“读段落”，而是做一次**全论文复盘 + 可复现路线**：把 7 个阶段、关键数据结构、每个阶段的输入/输出、可以直接从代码中定位的模块，以及若要把 AMF-Placer 思路移植到其它 placer 时最值得保留的机制整理成一张完整架构图。

End of Day 5. 本笔记只对论文正文明确支持的算法细节作确定性陈述；对论文未展开的 UTPlaceF / Rip-pleFPGA 内部机制未自行补公式。